

Project 02 - NACA Airfoil simulations with OpenFOAM

Exercise Sheet

Dr. Lambert Theisen & Dr. Georgii Oblapenko

1 Overview

In this mini-project you will implement a parameter study for a flow around an airfoil using OpenFOAM. You will need to write a python script that automatically generates the required meshes, runs the simulations, and post-processes the results. The final submission is a **research-style minipaper** of around **2-3 pages** (excluding figures) together with a **Git repository** containing code, project documentation, and all metadata needed to reproduce the results.

! Important

Deadline (end of that date): 05.07.2026

Submission: Via Moodle as repository link and PDF. Add us as members ([@lambert.theisen](#) / [@georgii.oblapenko](#)) to the project repository.

! Important

Software-engineering requirement. Treat this as a small research software project, not only as a report. Your work should live in a **Git repository** and be reproducible from the command line. Include code, and a short project documentation. Ideally, the result are two python files that can be run from the command line that will set up any necessary directories, set up the simulations, run them, and produce plots.

The first file performs a convergence study on the number of points used to represent the airfoil, the second file performs a parameter study of the impact of camber on the lift and drag coefficients. A [Jupyter notebook with python code](#) is provided as a starting point. The mesh generation is adapted from [curiosityFluidsAirfoilMesher.py](#) - an array of points is passed instead of a path to file containing the airfoil surface points.

1.1 Learning Outcomes

After completing this project, you should be able to:

- set up OpenFOAM and run basic testcases
- use various OpenFOAM utilities
- use Python scripts to automate workflows of executables that do not necessarily have a Python interface

1.2 Specific tasks

Specifically, in this project, you will be studying the impact of airfoil camber on the lift and drag coefficients for a 0 angle of attack. An example set up is provided further below, but it is for a fixed camber value and number of discretization points and does not produce any plots. You are expected to

- For a single camber value, perform a convergence study with respect to the number of points used to discretize the surface, the script should produce a convergence plot
- Having obtained a reasonable number of points for the discretization, verify for how long a simulation should run in order to achieve convergence in the lift and drag coefficients as well as the residuals
- Now having a reasonable number of points for the discretization **and** a suitable simulation time, for a range of camber values, perform a parameter study of the impact of camber on the lift and drag coefficients, varying camber (“M”) from 0 to 8. The report should have a plot of lift and drag coefficients as functions of the camber.

Bonus task: try running meshing on a highly-cambered airfoil with very few discretization points. Checkmesh should fail. Why is that? Inspect the mesh in ParaView. What is causing this behaviour?

2 NACA Airfoils

NACA airfoils are a set of standardized parameterized airfoil shapes (with chord length 1). Most common are the 4-digit ones (i.e. NACA2412); for many specific profiles experimental and resolved CFD data is openly available. This allows for solver validation/verification. The 4 digits are MP TT, and correspond to

- M: maximum camber times 100
- P: location of maximum camber times 10
- TT: maximum thickness times 100

Symmetric airfoils have M=0 and P=0, and the thickness is given by the last two digits. The airfoil thickness is given as

$$y_t = 5t [0.2969\sqrt{x} - 0.1260x - 0.3516x^2 + 0.2843x^3 - 0.1015x^4],$$

the mean camber line is computed as

$$y_c = \begin{cases} \frac{m}{p^2} (2px - x^2) & \text{for } 0 \leq x \leq p \\ \frac{m}{(1-p)^2} ((1-2p) + 2px - x^2) & \text{for } p \leq x \leq 1 \end{cases},$$

its slope is therefore given by

$$\frac{dy_c}{dx} = \begin{cases} \frac{2m}{p^2} (p - x) & \text{for } 0 \leq x \leq p \\ \frac{2m}{(1-p)^2} (p - x) & \text{for } p \leq x \leq 1 \end{cases},$$

and the coordinates of the upper (U) and lower (L) surfaces are

$$\theta = \arctan \left(\frac{dy_c}{dx} \right),$$

$$x_U = x - y_t \sin \theta, \quad y_U = y_c + y_t \cos \theta,$$

$$x_L = x + y_t \sin \theta, \quad y_L = y_c - y_t \cos \theta.$$

3 Example setup using curiosityFluidsAirfoilMesher.py

Below, is an example python code that

- sets up a single directory for the case
- generates a discretized surface
- produces a mesh via curiosityFluidsAirfoilMesher.py
- creates necessary parameter files for use with OpenFOAM
- runs OpenFOAM

```

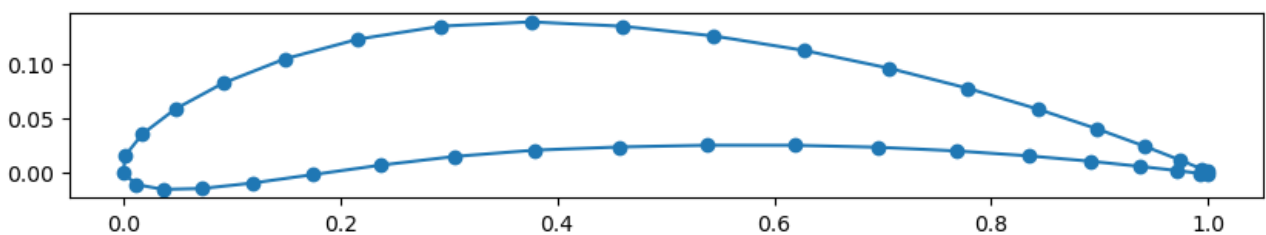
1  import os
2  import numpy as np
3  from matplotlib import pyplot as plt
4  import math
5  from pathlib import Path
6  import subprocess
7  import pandas as pd
8
9  # set up directories
10 os.makedirs("openfoam_naca/constant", exist_ok=True)
11 os.makedirs("openfoam_naca/system", exist_ok=True)
12
13 # write empty file for use with paraview
14 with open("openfoam_naca/case.foam", "w") as f:
15     f.write("")
16
17
18 # function to generate airfoil, plot it
19
20 def generate_naca4(digits, chord=1.0, points=100):
21     """Generates 2D coordinates for a 4-digit NACA airfoil."""
22     m = int(digits[0]) / 100.0 # Max camber
23     p = int(digits[1]) / 10.0  # Position of max camber
24     t = int(digits[2:]) / 100.0 # Max thickness
25
26     # Use cosine spacing to cluster more points at the leading edge for a smoother curve
27     beta = np.linspace(0, np.pi, points)
28     x = (1 - np.cos(beta)) / 2
29
30     # Thickness equation
31     # (Note: -0.1015 leaves a tiny trailing edge gap, standard for NACA.
32     # Shapely will automatically close this gap when making the polygon).
33     yt = 5 * t * (0.2969 * np.sqrt(x) - 0.1260 * x - 0.3516 * x**2 + 0.2843 * x**3 - 0.1015 *
34
35     # Camber line calculations
36     yc = np.zeros_like(x)
37     dyc_dx = np.zeros_like(x)

```

```

21
22     if p > 0:
23         # Ahead of max camber
24         front = x <= p
25         yc[front] = (m / p**2) * (2 * p * x[front] - x[front]**2)
26         dyc_dx[front] = (2 * m / p**2) * (p - x[front])
27
28         # Behind max camber
29         back = x > p
30         yc[back] = (m / (1 - p)**2) * ((1 - 2 * p) + 2 * p * x[back] - x[back]**2)
31         dyc_dx[back] = (2 * m / (1 - p)**2) * (p - x[back])
32
33     theta = np.arctan(dyc_dx)
34
35     # Upper surface coordinates
36     xu = x - yt * np.sin(theta)
37     yu = yc + yt * np.cos(theta)
38
39     # Lower surface coordinates
40     xl = x + yt * np.sin(theta)
41     yl = yc - yt * np.cos(theta)
42
43     # Combine into one continuous loop (Trailing Edge -> Leading Edge -> Trailing Edge)
44     x_coords = np.concatenate([xu[:-1], xl[1:]])
45     y_coords = np.concatenate([yu[:-1], yl[1:]])
46
47     # Scale by chord length
48     return np.column_stack((x_coords, y_coords)) * chord
49
50 airfoil_code = "8412"
51 chord_length = 1.0 # 1 meter long
52 n_surface_points = 20
53
54 discretized_surface_points = generate_naca4(airfoil_code, chord=chord_length, points=n_surface
55
56 ax = plt.subplots(1, 1, figsize=(10, 5))[1]
57
58 ax.plot(discretized_surface_points[:, 0], discretized_surface_points[:, 1], 'o-', label=f'NACA
59 ax.set_aspect("equal")
60 plt.show()

```



```

print(discretized_surface_points.shape)

```



```

purgeWrite      0;
writeFormat     ascii;
writePrecision  6;
writeCompression off;
timeFormat      general;
timePrecision   6;
runTimeModifiable true;

functions
{
    computeLiftDrag
    {
        type          forceCoeffs;          // Must be an unquoted word
        libs          ("libforces.so");     // Shared libraries CAN be a string list
        writeControl   timeStep;
        writeInterval  1;

        patches       (airfoil);            // Name of your boundary patch in patches/polyMesh
        p              p;                    // Pressure field name
        U              U;                    // Velocity field name

        // For incompressible flows (using foamRun with incompressibleFluid module)
        rho            rhoInf;
        rhoInf         {rho_inf}; // reference density

        CofR           (0 0 0);              // Center of Rotation (for moment coefficients)

        // Direction vectors (Adjust these based on your coordinate system orientation!)
        liftDir        (0 1 0);              // If lift is pointing up the Y-axis
        dragDir        (1 0 0);              // If drag is pointing along the X-axis
        pitchAxis      (0 0 1);              // Pitch axis (Z-axis)

        // Reference values used to normalize forces into Cd and Cl
        magUInf        {u_inf};              // Freestream velocity magnitude
        lRef           1.0;                  // Reference length (e.g., chord length of airfoil)
        Aref           {area};              // Planform Area (Chord 1.0m * Span 0.1m)
    }
}

residuals
{
    {
        type          residuals;
        libs          ("libutilityFunctionObjects.so");
        writeControl   timeStep;
        writeInterval  1;
        fields         (U p);
    }
}

// ***** //
"""

```

```

dict_path = os.path.join(system_dir, "controlDict")
with open(dict_path, "w") as f:
    f.write(content)

return dict_path

create_control_dict("openfoam_naca", rho_inf=rho_freestream, u_inf=u_freestream)

```

```
# run meshing
```

```

OPENFOAM_IMAGE = "microfluidica/openfoam:13"
PROJECT_DIR = Path.cwd().resolve()
case_dir = PROJECT_DIR / "openfoam_naca"

```

```
def foam_cmd(*args, workdir="/work"):
```

```

    return [
        "docker",
        "run",
        "--rm",
        "-v",
        f"{PROJECT_DIR}:/work",
        "-w",
        workdir,
        OPENFOAM_IMAGE,
        *args,
    ]

```

```
subprocess.run(foam_cmd("blockMesh", "-case", "openfoam_naca"))
```

```
# check mesh
```

```
subprocess.run(foam_cmd("checkMesh", "-case", "openfoam_naca"))
```

```
def create_momentum_transport_sa(case_dir):
```

```

    constant_dir = os.path.join(case_dir, "constant")
    os.makedirs(constant_dir, exist_ok=True)

```

```
# Using double curly braces {{ }} to escape them inside the Python f-string
```

```

content = f"""/*-----*- C++ -*-----
=====
\\\\\\      / F ield      | OpenFOAM: The Open Source CFD Toolbox
\\\\\\      / O peration   | Website:  https://openfoam.org
\\\\\\      / A nd         | Version:  13
\\\\\\      / M anipulation |
\\*-----*-
FoamFile
{{
    format      ascii;
    class       dictionary;
    location    "constant";
    object      momentumTransport;

```

```

}}
// * * * * *

simulationType  RAS;

RAS
{
    model          SpalartAllmaras;
    turbulence      on;
    printCoeffs     on;
}

// *****
"""

dict_path = os.path.join(constant_dir, "momentumTransport")
with open(dict_path, "w") as f:
    f.write(content)

    return dict_path

create_momentum_transport_sa("openfoam_naca")

```

```

def generate_openfoam_fvschemes_fvsolution(case_dir, steady_state=True):
    """
    Generates fvSchemes and fvSolution files tailored for OpenFOAM 13's
    modular framework (incompressibleFluid) with the Spalart-Allmaras model.
    """
    system_dir = os.path.join(case_dir, "system")

    # Determine schemes based on time-dependence
    ddt_scheme = "steadyState" if steady_state else "Euler"

    # -----
    # 1. Define fvSchemes Content
    # -----
    fv_schemes = f"""/*-----*- C++ -*-----
=====
\\\\\\      /  F ield          | OpenFOAM: The Open Source CFD Toolbox
\\\\\\      /  O peration       | Website:  https://openfoam.org
\\\\\\      /  A nd              | Version:  13
\\\\\\      /  M anipulation     |
\\*-----*-
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    object       fvSchemes;
}
// * * * * *

```



```

ddtSchemes
{{
    default          {ddt_scheme};
}}

gradSchemes
{{
    default          Gauss linear;
    grad(U)          Gauss linear;
}}

divSchemes
{{
    default          none;
    div(phi,U)       Gauss linearUpwind grad(U);
    div(phi,nuTilda) Gauss upwind;
    div((nuEff*dev2(T(grad(U)))) Gauss linear;
}}

laplacianSchemes
{{
    default          Gauss linear corrected;
}}

interpolationSchemes
{{
    default          linear;
}}

snGradSchemes
{{
    default          corrected;
}}

wallDist
{{
    method          meshWave;
}}

// ***** //
"""

# -----
# 2. Define fvSolution Content
# -----
n_outer_correctors = 1 if steady_state else 2

# Relaxation factors are critical for numerical stability in steady-state runs
relaxation_factors = ""relaxationFactors
{

```

```

fields
{
    p                0.3;
}
equations
{
    U                0.7;
    nuTilda          0.7;
}
}"" if steady_state else ""

fv_solution = f""/*-----*- C++ -*-----*/
=====
\\\\\\ / F ield      | OpenFOAM: The Open Source CFD Toolbox
\\\\\\ / O peration   | Website:  https://openfoam.org
\\\\\\ / A nd         | Version:  13
\\\\\\ / M anipulation |
\\*-----*/
FoamFile
{{
    version      2.0;
    format       ascii;
    class        dictionary;
    object       fvSolution;
}}
// * * * * *

solvers
{{
    p
    {{
        solver      GAMG;
        tolerance    1e-06;
        relTol      0.01;
        smoother     GaussSeidel;
    }}

    pFinal
    {{
        $p;
        tolerance    1e-06;
        relTol      0;
    }}

    "(U|nuTilda)"
    {{
        solver      PBiCGStab;
        preconditioner DILU;
        tolerance    1e-05;
        relTol      0.1;
    }}
}}

```

```

        "(U|nuTilda)Final"
        {{
            solver          PBiCGStab;
            preconditioner   DILU;
            tolerance        1e-05;
            relTol           0;
        }}
    }}

PIMPLE
{{
    nOuterCorrectors {n_outer_correctors};
    nCorrectors      2;
    nNonOrthogonalCorrectors 0;
}}

{relaxation_factors}

// ***** //
"""

# Write files out to the system folder
with open(os.path.join(system_dir, "fvSchemes"), "w") as f:
    f.write(fv_schemes)

with open(os.path.join(system_dir, "fvSolution"), "w") as f:
    f.write(fv_solution)

generate_openfoam_fvschemes_fvsolution("openfoam_naca")

```

```

def generate_physical_properties(case_dir, nu_value=1.5e-5):
    """
    Generates a physicalProperties file for incompressible flow.
    """
    constant_dir = os.path.join(case_dir, "constant")

    file_content = f"""/*-----*- C++ -*-----
=====
\\\\\\ / F i e l d | OpenFOAM: The Open Source CFD Toolbox
\\\\\\ / O p e r a t i o n | Website: https://openfoam.org
\\\\\\ / A n d | Version: 13
\\\\\\ / M a n i p u l a t i o n |
\\*-----*/

FoamFile
{{
    format      ascii;
    class       dictionary;
    location     "constant";
    object       physicalProperties;
}}

```

```

// * * * * *

viscosityModel    constant;

nu                [0 2 -1 0 0 0 0] {nu_value};

// *****

"""

    file_path = os.path.join(constant_dir, "physicalProperties")
    with open(file_path, "w") as f:
        f.write(file_content)

generate_physical_properties("openfoam_naca", nu_value=1.5e-5)

def generate_0_directory(case_dir, u_freestream, rho_freestream, nu=1.5e-5):
    """
    Generates initial and boundary condition field files in the 0/ directory
    for an OpenFOAM 13 incompressible Spalart-Allmaras airfoil simulation.
    """
    zero_dir = os.path.join(case_dir, "0")
    os.makedirs(zero_dir, exist_ok=True)

    # --- Turbulence Math for Spalart-Allmaras Freestream values ---
    # Standard guideline: nuTilda_freestream is 3 to 5 times kinematic viscosity
    nu_tilda_freestream = 3.0 * nu

    # Calculate initial nut from nuTilda using the SA model formulation
    cv1 = 7.1
    chi = nu_tilda_freestream / nu
    fv1 = (chi**3) / (chi**3 + cv1**3)
    nut_freestream = nu_tilda_freestream * fv1

    print(f"[+] Physical Constants & Computed Freestream Targets:")
    print(f"    - Inflow Velocity: {u_freestream} m/s")
    print(f"    - Freestream Density: {rho_freestream} kg/m³ (Used for reference/forces)")
    print(f"    - Kinematic Viscosity (nu): {nu} m²/s")
    print(f"    - Freestream nuTilda: {nu_tilda_freestream:.2e} m²/s")
    print(f"    - Freestream nut: {nut_freestream:.2e} m²/s\n")

    # -----
    # 1. Field: U (Velocity Field)
    # -----
    # -----
    # Velocity (U)
    # -----
    u_content = f"""/*-----*- C++ -*-----
=====
\\\\\\      /  F ield          | OpenFOAM: The Open Source CFD Toolbox
\\\\\\      /  O peration        | Website:  https://openfoam.org
\\\\\\      /  A nd              | Version:  13

```

```

    \\\\/      M anipulation  |
\\*-----*/
FoamFile
{{
    version      2.0;
    format        ascii;
    class         volVectorField;
    location      "0";
    object        U;
}}
// * * * * *

dimensions      [0 1 -1 0 0 0 0];

internalField    uniform ({u_freestream} 0 0);

boundaryField
{{
    farfield
    {{
        type              freestream;
        freestreamValue    uniform ({u_freestream} 0 0);
    }}

    airfoil
    {{
        type              noSlip;
    }}

    frontAndBack
    {{
        type              empty;
    }}
}}

// *****
"""

# -----
# Kinematic Pressure (p = P_physical / rho)
# -----
p_content = f""/*-----*- C++ -*-----
=====
\\\/      / F ield      | OpenFOAM: The Open Source CFD Toolbox
\\\/      / O peration   | Website:  https://openfoam.org
\\\/      / A nd         | Version:   13
\\\/      / M anipulation |
\\*-----*/
FoamFile
{{
    version      2.0;

```

```

    format      ascii;
    class       volScalarField;
    location    "0";
    object      p;
}}
// * * * * *

dimensions      [0 2 -2 0 0 0 0];

internalField   uniform 0;

boundaryField
{
    farfield
    {
        type          freestreamPressure;
        freestreamValue uniform 0;
    }

    airfoil
    {
        type          zeroGradient;
    }

    frontAndBack
    {
        type          empty;
    }
}

// *****
"""

# -----
# Modified Turbulent Viscosity (nuTilda)
# -----
nutilda_content = f"""/*-----*- C++ -*-----
=====
\\\\\\      /  F ield      | OpenFOAM: The Open Source CFD Toolbox
\\\\\\      /  O peration    | Website:  https://openfoam.org
\\\\\\      /  A nd          | Version:  13
\\\\\\      /  M anipulation  |
\\*-----*-
FoamFile
{
    version      2.0;
    format       ascii;
    class        volScalarField;
    location     "0";
    object       nuTilda;
}

```

```

// * * * * *

dimensions      [0 2 -1 0 0 0 0];

internalField    uniform {nu_tilda_freestream};

boundaryField
{
    farfield
    {
        type      freestream;
        freestreamValue uniform {nu_tilda_freestream};
    }

    airfoil
    {
        type      fixedValue;
        value      uniform 0;
    }

    frontAndBack
    {
        type      empty;
    }
}

// ***** //
"""

# -----
# Turbulent Kinematic Viscosity (nut)
# -----
nut_content = f"""/*-----*- C++ -*-----
=====
\\\\\\ / F ield      | OpenFOAM: The Open Source CFD Toolbox
\\\\\\ / O peration   | Website:  https://openfoam.org
\\\\\\ / A nd         | Version:  13
\\\\\\ / M anipulation |
\\*-----*/
FoamFile
{
    version      2.0;
    format       ascii;
    class        volScalarField;
    location     "0";
    object       nut;
}
// * * * * *

dimensions      [0 2 -1 0 0 0 0];

```

```

internalField    uniform {nut_freestream};

boundaryField
{
    farfield
    {
        type      calculated;
        value      uniform {nut_freestream};
    }

    airfoil
    {
        type      nutUSpaldingWallFunction;
        value      uniform 0;
    }

    frontAndBack
    {
        type      empty;
    }
}

// *****

"""

# Write files to disk
fields = {"U": u_content, "p": p_content, "nuTilda": nutilda_content, "nut": nut_content}
for field_name, script_content in fields.items():
    file_path = os.path.join(zero_dir, field_name)
    with open(file_path, "w") as f:
        f.write(script_content)

generate_0_directory("openfoam_naca", u_freestream, rho_freestream)

subprocess.run(foam_cmd("foamRun", "-case", "openfoam_naca"))

forcecoeffs_file = "openfoam_naca/postProcessing/computeLiftDrag/0/forceCoeffs.dat"

# Read the space-delimited file, ignoring the OpenFOAM header lines starting with '#'
# Column 0 is Time, Column 1 is Cd, Column 3 is Cl
df = pd.read_csv(forcecoeffs_file, sep=r'\s+', comment='#', header=None)

# Extract columns
time = df[0]
cd = df[2]
cl = df[3]

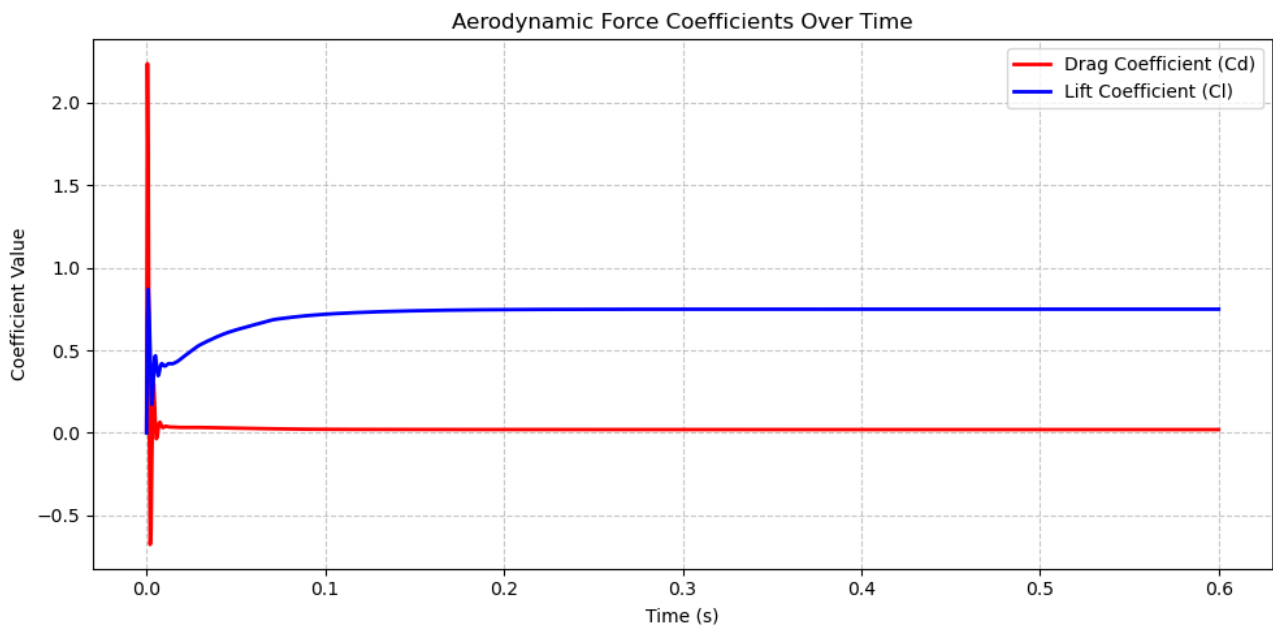
plt.figure(figsize=(10, 5))
plt.plot(time, cd, label="Drag Coefficient (Cd)", color="red", linewidth=2)
plt.plot(time, cl, label="Lift Coefficient (Cl)", color="blue", linewidth=2)

```



```
plt.title("Aerodynamic Force Coefficients Over Time")
plt.xlabel("Time (s)")
plt.ylabel("Coefficient Value")
plt.grid(True, linestyle="--", alpha=0.7)
plt.legend()

plt.tight_layout()
# plt.savefig("force_coefficients_plot.png")
plt.show()
```



```
residuals_file = "openfoam_naca/postProcessing/residuals/0/residuals.dat"

# Read the data, ignoring headers and comments
df = pd.read_csv(residuals_file, sep=r'\s+', comment='#', header=None)

plt.figure(figsize=(10, 6))

time = df[0]
r_ux = df[1]
r_uy = df[2]
r_p = df[3]

plt.plot(time, r_p, label='Pressure', color='black', alpha=0.8)
plt.plot(time, r_ux, label='Velocity Ux', color='blue', alpha=0.8)
plt.plot(time, r_uy, label='Velocity Uy', color='green', alpha=0.8)

# Set the Y-axis to logarithmic to view convergence properly
plt.yscale('log')

plt.title("Solver Residuals Over Time")
plt.xlabel("Time (s)")
plt.ylabel("Residual (Log Scale)")
```

```
plt.grid(True, which="both", linestyle="--", alpha=0.5)
plt.legend()

plt.tight_layout()
# plt.savefig("residuals_plot.png")
plt.show()
```

